

1. Frontend: El ecosistema Angular

Al elegir Angular, estamos optando por un marco de trabajo estructurado que facilita el mantenimiento.

- **Framework: Angular (v17+)** utilizando **TypeScript**.
- **Librería de Mapas: Leaflet.js** con el paquete **@asymmetrik/ngx-leaflet**. Es la mejor opción gratuita para manejar capas de mapas y polilíneas sin las restricciones de pago de Google Maps.
- **Interfaz de Usuario (UI): Angular Material**. Proporciona componentes listos como el **Sidenav** para el menú, **Cards** para los hospitales y **Bottom Sheets** para la versión móvil.
- **Gestión de Estado: RxJS**. Es fundamental para manejar de forma asíncrona la respuesta del mapa y los datos del backend.

2. Backend: Robustez con Java

Para alumnos de DAM, lo más coherente es seguir la línea de aprendizaje de Java.

- **Framework: Spring Boot**. Es el estándar para crear APIs REST rápidas y seguras.
- **Documentación: Swagger / OpenAPI**. Imprescindible para que el equipo de frontend sepa qué endpoints consumir sin dudas.
- **Seguridad: Spring Security** con **JWT (JSON Web Tokens)** para gestionar el login de usuarios registrados.

3. Base de Datos: El motor geográfico

Este es el componente más crítico del proyecto.

- **Motor: PostgreSQL**.
- **Extensión Espacial: PostGIS**. Es gratuita y permite realizar la lógica central del proyecto: calcular qué hospital está más cerca de un punto exacto de la carretera mediante funciones SQL geográficas.

4. Servicios Externos (Gratuitos)

Para que el mapa funcione con datos reales sin coste:

- **Tiles (Teselas del mapa): OpenStreetMap (OSM)**.
- **Geocodificación: Nominatim** (para buscar "Badajoz" y obtener sus coordenadas).
- **Cálculo de Rutas: OSRM (Open Source Routing Machine)**. Proporciona el tiempo de viaje real por carretera, no en línea recta.

GUÍA TÉCNICA DE DESARROLLO: PROYECTO "GOLDEN HOUR"

Esta guía detalla el ecosistema de herramientas necesarias para construir la aplicación web de seguridad sanitaria utilizando tecnologías profesionales, gratuitas y de código abierto.

1. Requisitos de Software (Instalación Inicial)

Antes de comenzar la codificación, cada puesto de trabajo debe tener instalado el siguiente software de base:

- **Node.js (Versión LTS):** Entorno de ejecución para Angular y gestión de dependencias npm.
- **Java Development Kit (JDK) 17 o 21:** Necesario para el desarrollo del backend con Spring Boot.
- **PostgreSQL 15+:** Motor de base de datos relacional.
- **Docker Desktop (Recomendado):** Para levantar servicios como la base de datos de forma rápida y aislada.
- **Entornos de Desarrollo (IDE):**
 - **Frontend:** Visual Studio Code (con extensiones: *Angular Language Service* y *ESLint*).
 - **Backend:** IntelliJ IDEA (Community Edition) o Spring Tool Suite (STS).
- **Postman / Insomnia:** Herramientas para testear las peticiones a la API REST.

2. Configuración del Frontend (Angular + Leaflet)

El frontend se desarrollará como una **Single Page Application (SPA)** escalable.

Comandos de instalación:

Ejecuta estos comandos en tu terminal para inicializar el proyecto y sus dependencias críticas:

Bash

1. Instalar el CLI de Angular

```
npm install -g @angular/cli
```

2. Crear el nuevo proyecto

```
ng new golden-hour-front --routing --style=scss
```

3. Entrar a la carpeta e instalar librerías de mapas y UI

```
cd golden-hour-front
```

```
npm install leaflet @asymmetrik/ngx-leaflet
```

```
npm install @angular/material @angular/cdk @angular/animations
```

4. Instalar tipos de Leaflet para soporte en TypeScript

```
npm install --save-dev @types/leaflet
```

3. Configuración del Backend (Spring Boot)

El backend gestionará la lógica de negocio y la conexión con la base de datos geográfica.

- **Generador:** Usa [Spring Initializr](#).
- **Dependencias Clave:**
 - **Spring Web:** Para crear los controladores de la API REST.
 - **Spring Data JPA:** Para la persistencia de datos.
 - **PostgreSQL Driver:** Para la conexión con la base de datos.
 - **Lombok:** Para reducir código repetitivo (Getters/Setters).
 - **Spring Security + jwt:** Para el login y cifrado de tokens.
 - **Springdoc-openapi (Swagger):** Para documentar la API automáticamente.

4. Base de Datos Geográfica (PostGIS)

Sin PostGIS, la aplicación no podrá calcular distancias reales por carretera de forma eficiente.

Opción Rápida con Docker:

Bash

```
docker run --name gh-db -e POSTGRES_PASSWORD=dam_pass -p 5432:5432 -d  
postgis/postgis
```

Activación Manual:

Si instalas PostgreSQL de forma local, abre tu consola SQL y ejecuta:

SQL

```
-- Activa las funciones de cálculo espacial (geometría y geografía)  
CREATE EXTENSION postgis;
```

5. Integración de Servicios Externos (Gratuitos)

Para que el mapa tenga datos sin pagar cuotas de suscripción, configurad vuestros servicios con estas fuentes:

Servicio	Proveedor	Uso en el código
Teselas (Mapas)	OpenStreetMap	<code>https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png</code>
Rutas y Tiempos	OSRM Project	<code>http://router.project-osrm.org/route/v1/driving/</code>
Buscador Direcciones	Nominatim	<code>https://nominatim.openstreetmap.org/search?format=json</code>

Recomendaciones:

1. **Modelo de Datos en Angular:** Cread interfaces sólidas para los Hospitales y las Rutas. No uséis el tipo `any`.
2. **Cálculo de la Golden Hour:** El backend debe devolver los tramos de la ruta con un atributo de "riesgo". El frontend (Angular) solo debe preocuparse de pintar esos tramos con el color que le indique el servidor.
3. **Z-Index en CSS:** Recordad que para que el buscador flote sobre el mapa de Leaflet, debéis asignar al contenedor de búsqueda un `z-index` superior a 1000.
4. **CORS:** No olvidéis configurar el `@CrossOrigin` en vuestros controladores de Spring Boot para permitir peticiones desde el puerto de Angular (`4200`).